

Computer Science & IT

Compiler Design

Code Optimisation

Lecture No 02



Topics to be Covered



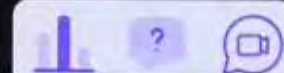
Topic

ICG

Topic

Topic

90% → Syllabus over :-



$$S \rightarrow S_1 S_2 C \{ S.t = S_1.t - S_2.t \}$$

$$/a \quad \{ S.t = 5 \}$$

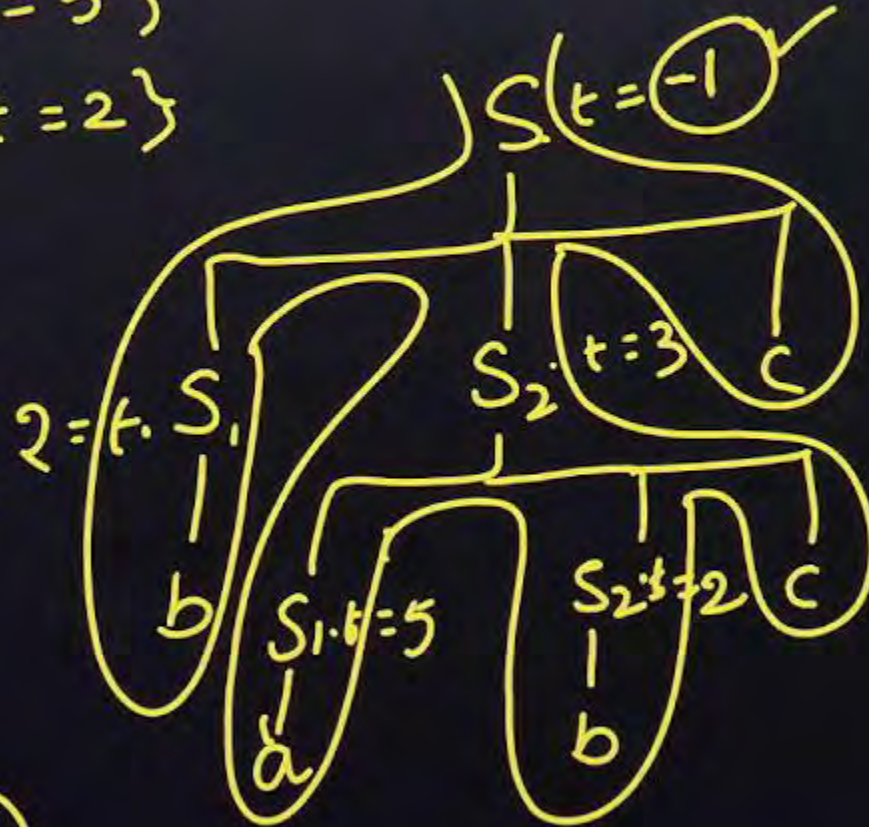
$$/b \quad \{ S.t = 2 \}$$

babcc $\rightarrow$ o/p

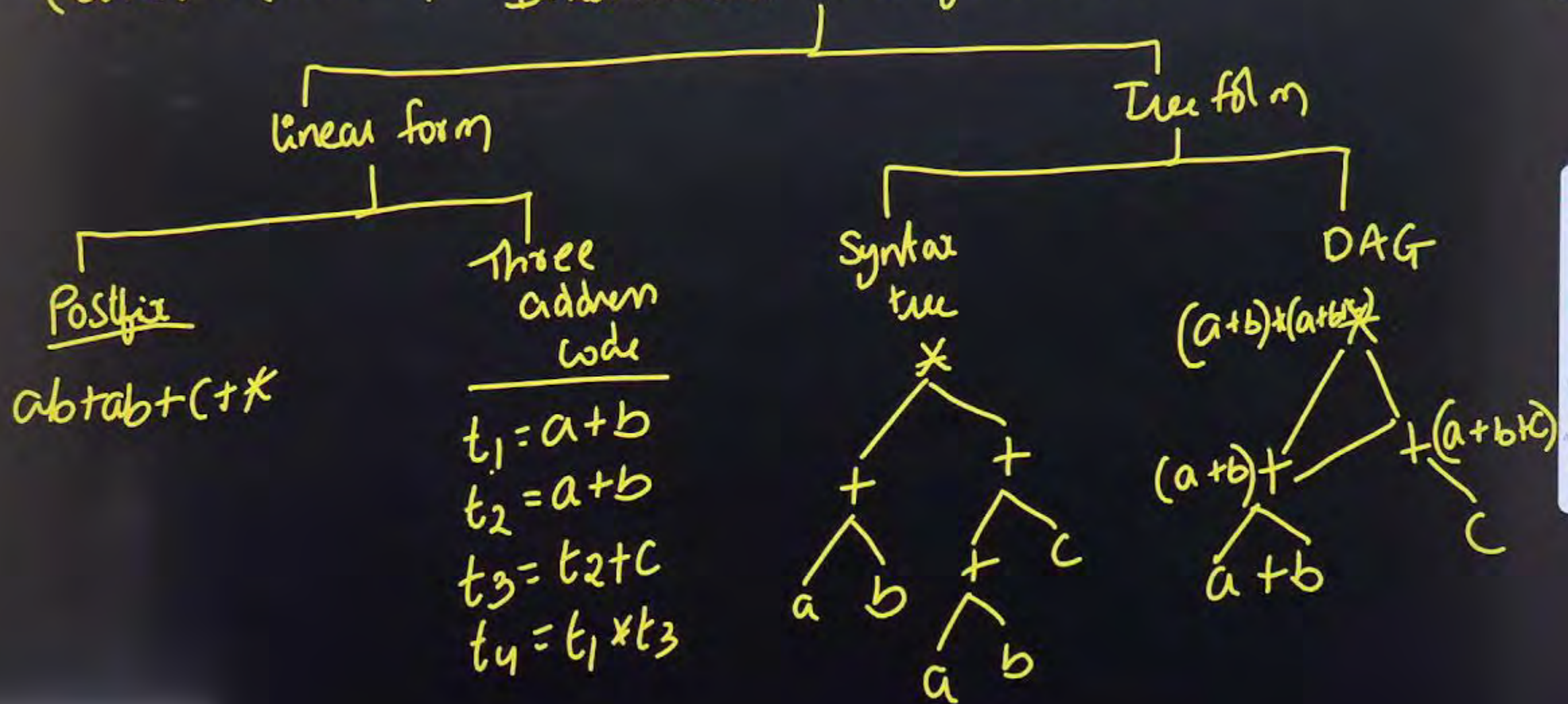
LA, SA, SA, (ICG) ✓

90% - CD

Parser



$(a+b) * (a+b+c)$ Intermediate code generation



Types of 3 address code:-

- 1) $x = y \text{ op } z$
- 2) $x = \text{op } z$
- 3) $x = y$
- 4) $\text{if } \underline{x} \text{ rel op } y \text{ goto } \underline{L}$
- 5) $\text{goto } L$
- 6) $A[i] = x$
 $y = A[i]$
- 7) $x = *P$
 $y = \&x$

2) function calling:-

Param P_1
Param P_2
Param P_3
⋮
Param P_n

$t = \text{Call } \check{f}, \check{n}$

9) return

$$-(a+b)*(c+d) + (a+b+c)$$

- 1) $t_1 = a+b$
- 2) $t_2 = -t_1$
- 3) $t_3 = c+d$
- 4) $t_4 = t_2 * t_3$
- 5) $t_5 = a+b$
- 6) $t_6 = t_5 + c$
- 7) $t_7 = t_4 + t_6$

Quadruples:-

	opr	op1	op2	result
1)	+	a	b	t_1
2)	-	t_1		t_2
3)	+	c	d	t_3
4)	*	t_2	t_3	t_4
5)	+	a	b	t_5
6)	+	t_5	c	t_6
7)	+	t_4	t_6	t_7

adv:- Statements can be moved around

Tuple

	opr	op1	op2
1)	+	a	b
2)	-	(1)	
3)	+	c	d
4)	*	(2)	(3)
5)	+	a	b
6)	+	(5)	c
7)	+	(4)	(6)

adv:- Space is not wasted
dis:- Statements cannot be moved

Indirect tuple:-

(i) (1)

(ii) (2)

(iii) (3)

(iv) (4)

(v) (5)

(vi) (6)

(vii) (7)

adv: Statements can be moved around

dis: Two memory accesses

3 min

gate 2005

Back patching:-

if $(a < b)$ then $t = 1$
else $t = 0$

3 add (i) if $(a < b)$ goto (i+3)

(i+1) $t = 0$

(i+2) goto (i+4)

(i+3) $t = 1$

(i+4)

Gate 2012:- $t_1=10$ $t_2=10$ $t_3=100$
 $(a < b)$ and $(c < d)$ or $(e < f)$

100) if $(a < b)$ goto 103

101) $t_1 = 0$

102) goto 104

103) $t_1 = 1$

104) if $(c < d)$ goto 107

105) $t_2 = 0$

106) goto 108

107) $t_2 = 1$

108) if $(e < f)$ goto 111

109) $t_3 = 0$

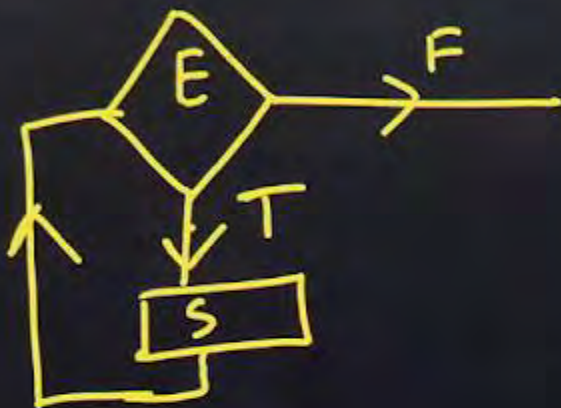
110) goto 112

111) $t_3 = 1$

112) $t_4 = t_2$ and t_3

113) $t_5 = t_4 \cup t_3$

while E do S



L: if (E == 0) goto L₁
S[✓]
goto L[✓]

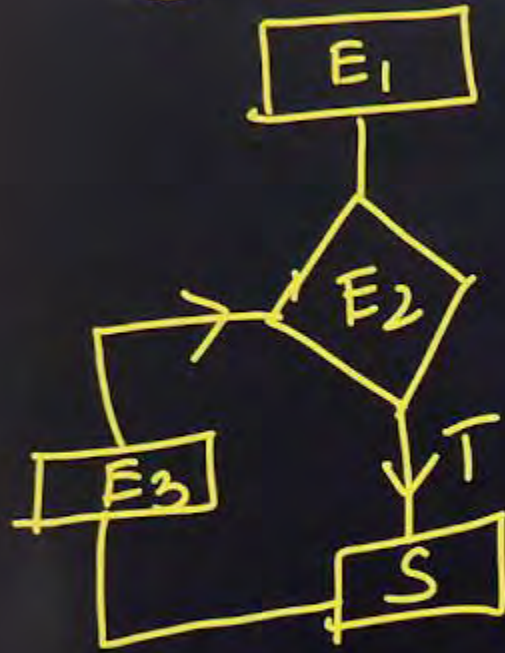
L₁:

L: if (E) goto L₁
goto last

L₁: S[✓]
goto L[✓]

last:

$fA(E_1; E_2; E_3)$
S



$fA(i=0; i<10; i++)$
 $a=b+c$

$L: \text{if } (i < 10) \text{ goto } L_1$
 goto last

$L_1: \left. \begin{array}{l} t_1 = b+c \\ a = t_1 \\ t_2 = i+1 \\ i = t_2 \end{array} \right\}$
 $\text{goto } L$

last —

```

Switch(i+j)
{
  case 1: a = b + c;
          break;
  case 2: p = q + r;
          break;
  default: x = y + z;
           break;
}

```

```

t = i + j
goto: test

```

```

L1: t1 = b + c
    a = t1
    goto last

```

```

L2: t2 = q + r
    p = t2
    goto last

```

```

L3: t3 = y + z
    x = t3
    goto last

```

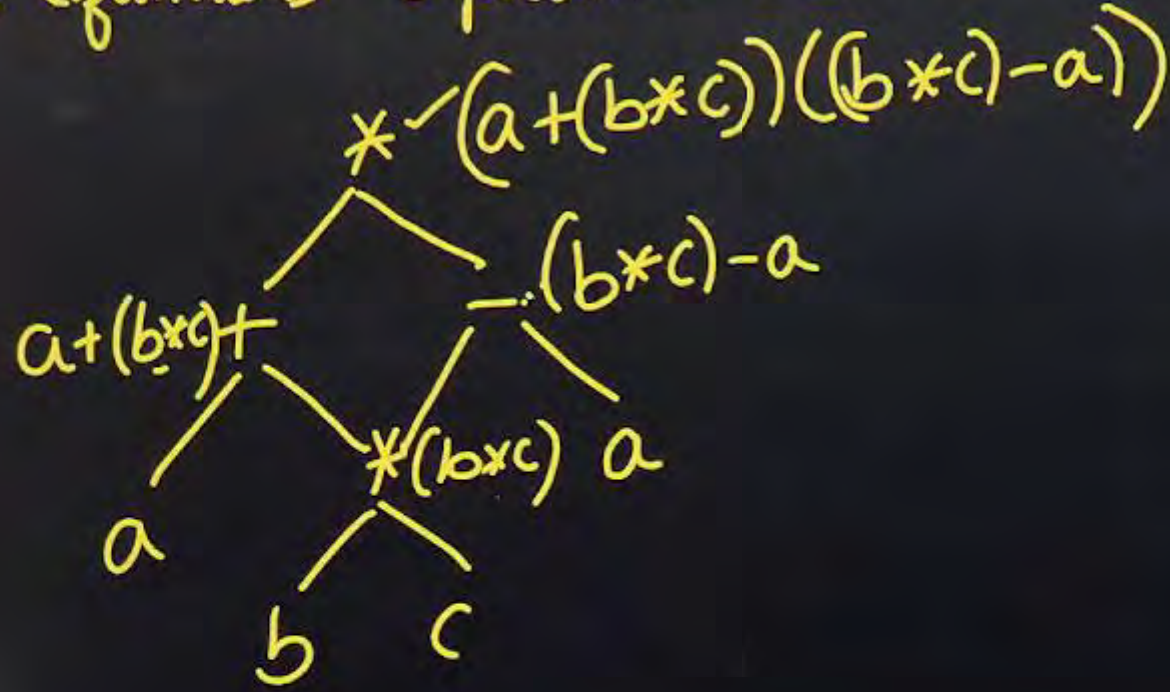
```

test: if t == 1 goto L1
      if (t == 2) goto L2
      goto t3

```

last:

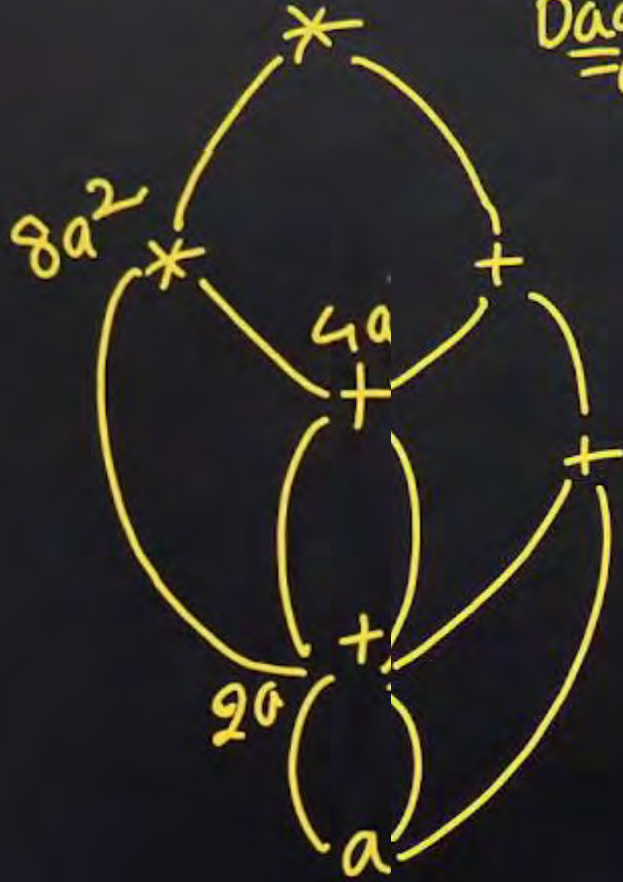
The equivalent expression for the following DAG.



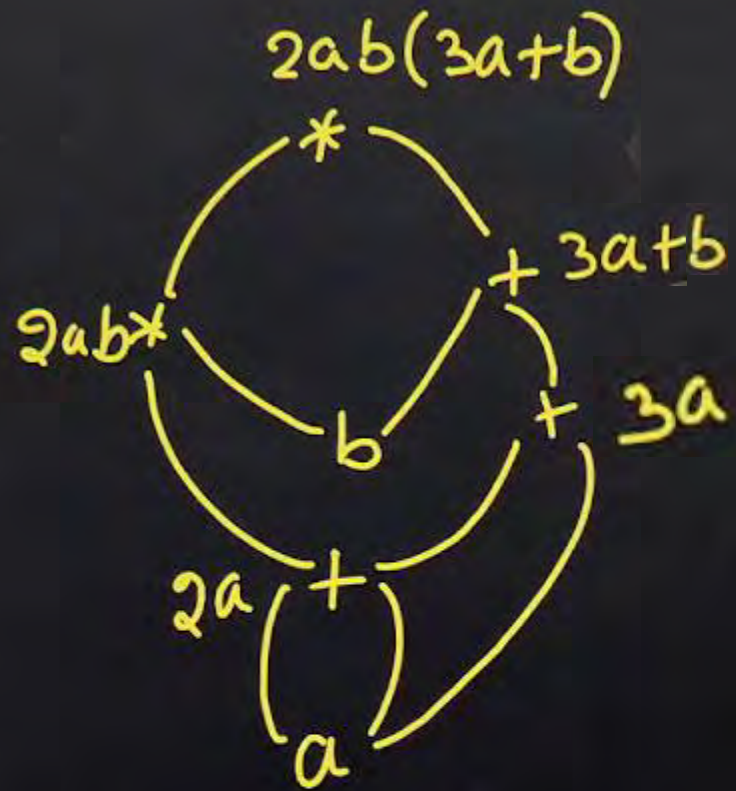
$$8a^2 \times 7a = 448$$

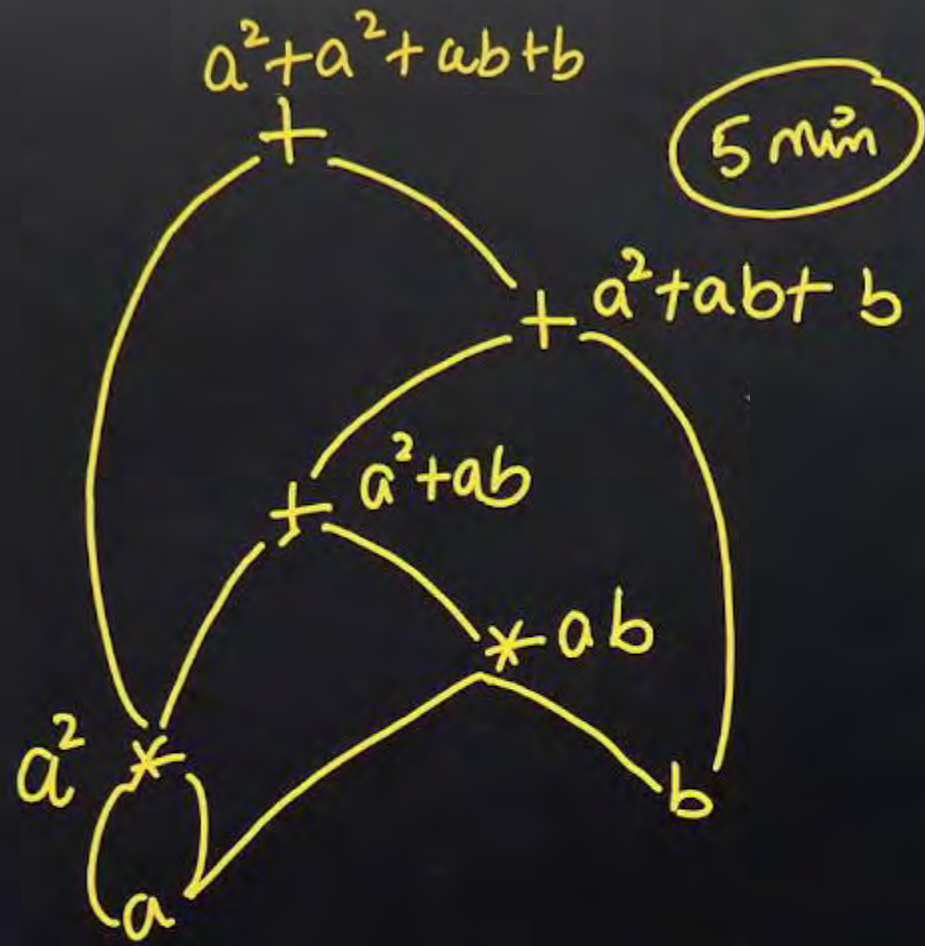
Dag

5 min



if $a=2$, then val = ?





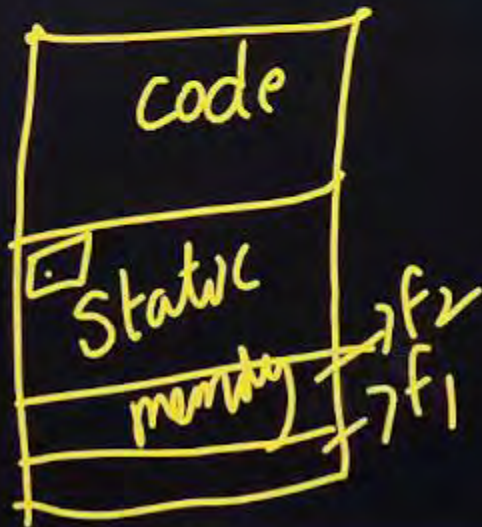
Runtime environments:-



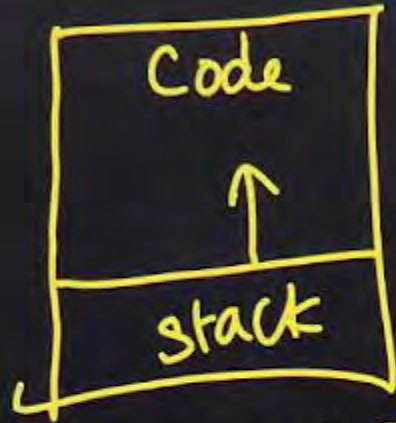
Storage allocation strategies:-

1) Static:-

- a) allocation is done at compile time
- b) Binding do not change at Run time
- c) one activation record (frame) per procedure



2) Stack:-



whenever a new activation begins, ^(frame) activation record is pushed on to the stack and whenever activation ends, activation record is popped off. local variables are bound to fresh memory.

Dis:- Local variables cannot be retained once activation ends.

Adv:- Recursion in

Heap:-



Allocation and deallocation can be done in any order during Runtime

Dis: Heap management is a overhead

Summary:-

- Activations can have
- (i) Permanent lifetime in case of static allocation
 - (ii) nested life time in case of stack



- (iii) Arbitrary life time in case of heap

Activation Record (frame)

Actual parameters
Returned values
Control link
Access link
Save n/c Status
Local data
Templates

1) Temporaries:-

Temporary variables arise from the evaluation of expressions

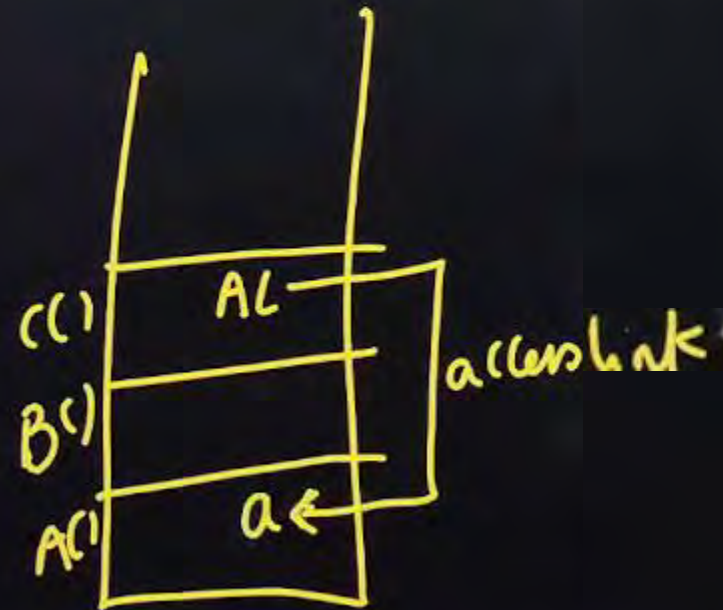
In case there is no space in registers, they will be stored in

activation records

2) Local data:- It belongs to the procedure whose activation record it is

3) Saved m/c Status:- It contains the information about the state of the m/c just before the call to the procedure.

4) Access link:- An access link may be needed to locate data needed by the called procedure but found elsewhere e.g. in another activation record



6) Returned values - Space for the return value of the called function

7) Actual parameter:-

The actual parameters are passed by calling the procedure
Commonly these values are placed in the activation record
rather than registers

Code optimization

15:40

machine independent

- 1) Loop optimisations
 - a) Code motion & frequency reduction
 - b) loop unrolling
 - c) loop jamming
- 2) Constant folding
- 3) Constant propagation
- 4) Redundancy elimination
- 5) Strength reduction

machine dependent

- 1) Register allocation
- 2) use of addressing modes
- 3) Peephole optimization

- a) Redundant load/store
- b) flow of control optimization
- c) Strength reduction
- d) use of m/c idiom

CO

Loop optimisations:-

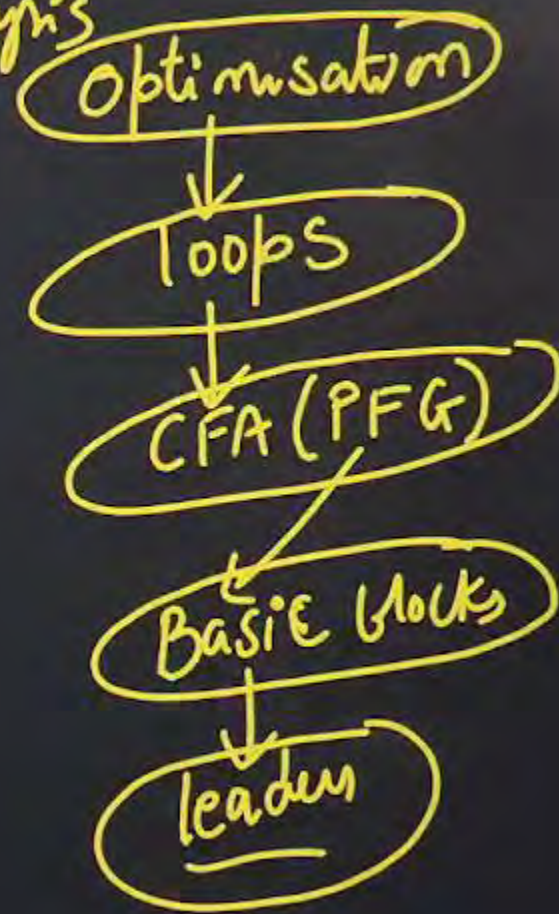
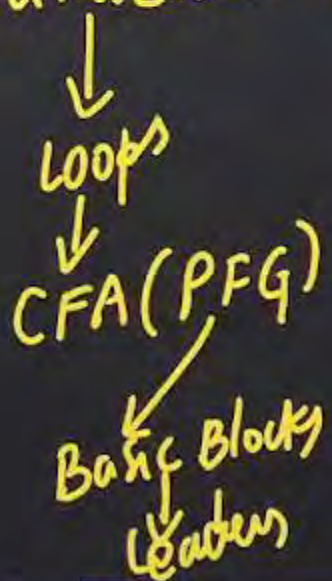
→ To apply loop optimisations, we must first detect the loops

→ For detecting the loops, we can use control flow analysis

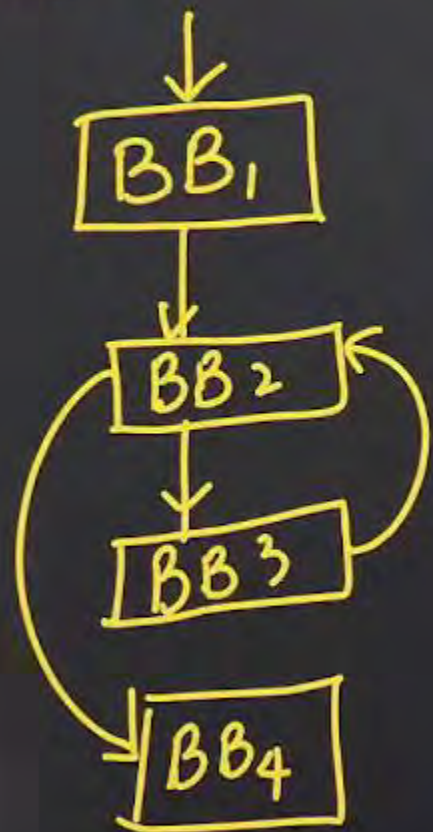
using program flow graph

→ To find PFG, we need to find basic blocks

optimisations



Given a program we will construct the program flow graph and examine how the control is flowing



A cycle in PFG is a loop in the program

Finding the Basic blocks:-

→ In order to find the basic blocks, we need to find the leaders in the program, then a basic block will start from one leader to the next leader but not including the next leader.

Identifying leaders in a basic block:-

- 1) I Statement is a leader
- 2) Statement that is target of Conditional or unconditional Statement is a leader
- 3) Statements that follow Conditional or unconditional Statement is a leader

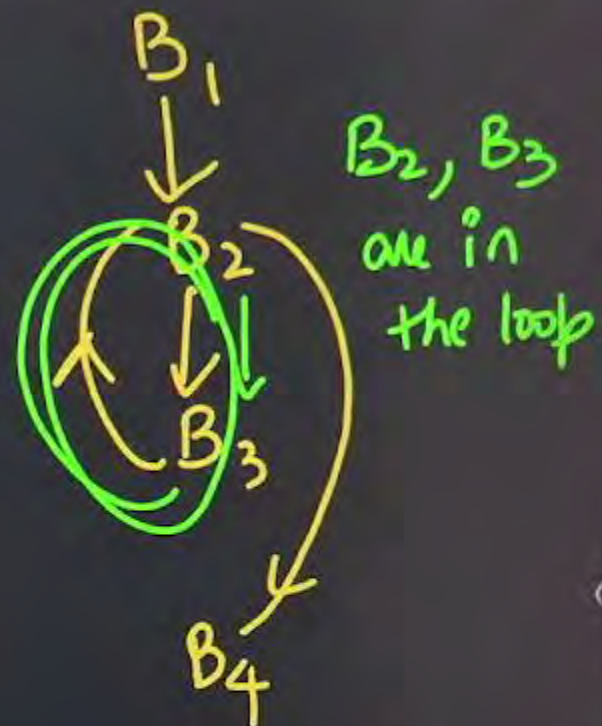
3 - add Code :-

* 1) $f = 1$
2) $i = 2$ B_1

* 3) if ($i > x$) goto 9. B_2

* 4) $t_1 = f * i$
5) $f = t_1$
6) $t_2 = i + 1$
7) $i = t_2$
8) goto 3 B_3

* 9) Goto Calling program B_4



Cycle in the CFG is loop in the program

```

fact(x)
{
  int f=1;
  for(i=2; i<=x; i++)
    f = f * i;
  return f;
}
  
```

Thank You